Diffusion Model on MNIST

Implementation and Evaluation Report

Denoising Diffusion Probabilistic Models (DDPM)

GitHub: https://github.com/TN108/DDPM

March 2026

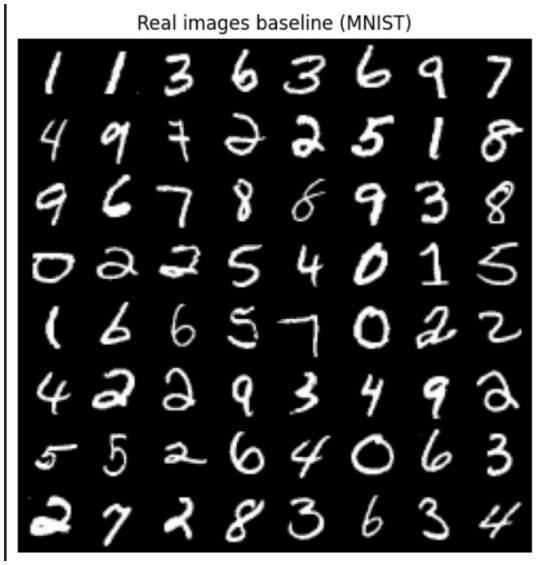

Figure 1: Real MNIST handwritten digit images (baseline)

1. Introduction

This project implements a Denoising Diffusion Probabilistic Model (DDPM) to generate handwritten digit images from the MNIST dataset. Diffusion models work by gradually adding noise to images and training a neural network to reverse this process. The model learns to predict the noise present in a noisy image and then removes it step by step to generate new images.

The goals of this project were to:

- Implement the forward and reverse diffusion processes
- Train a neural network to predict noise
- Generate images using reverse diffusion sampling
- Perform ablation experiments and evaluate model performance

2. Dataset and Preprocessing

The MNIST dataset was used, containing grayscale images of handwritten digits spanning classes 0 through 9.

Key dataset properties:

- Image size: 28 × 28 pixels
- Channels: 1 (grayscale)
- Classes: 10 digits (0–9)

Images were normalized to the range $x \in [-1, 1]$. This normalization stabilizes training and ensures the diffusion equations operate correctly. A dataloader was created to load mini-batches of images during training.

3. Diffusion Process

3.1 Forward Diffusion

In the forward process, Gaussian noise is gradually added to an image over T timesteps. The noisy image x_t at timestep t is computed via the closed-form equation:

```
x_t = sqrt(alpha_bar_t) · x_0 + sqrt(1 - alpha_bar_t) · ε
```

where x_0 is the original image, ε is Gaussian noise sampled from N(0, I), and alpha_bar_t controls the noise level at timestep t. As t increases, the image becomes progressively noisier until it approximates pure Gaussian noise.

The plots below show the alpha_bar(t) schedule and the corresponding Signal-to-Noise Ratio (SNR) over timesteps, illustrating how signal is preserved and then lost during the forward process:

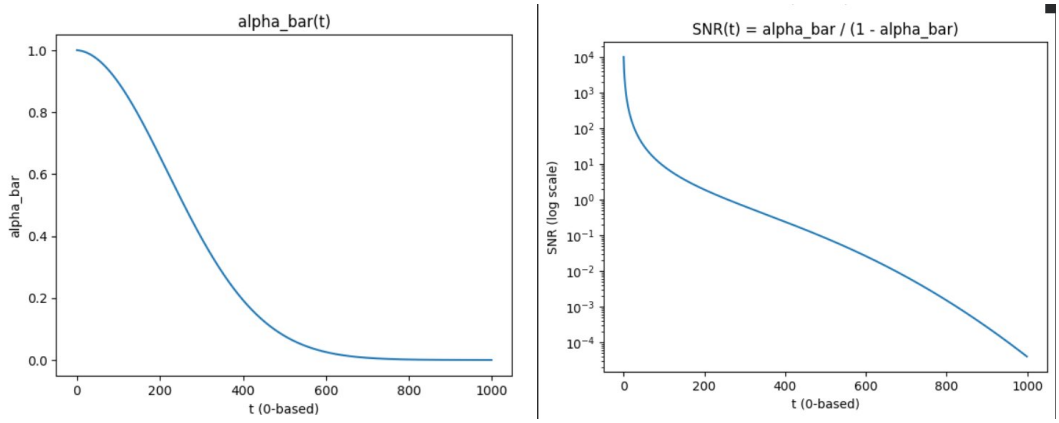

Figure 2–3: alpha_bar(t) schedule (left) and SNR(t) on log scale (right)

3.2 Noising Progression

The figure below illustrates how a sample digit image (the digit "3") degrades through the forward diffusion process at timesteps t = 0, 50, 200, 500, 800, and 999:

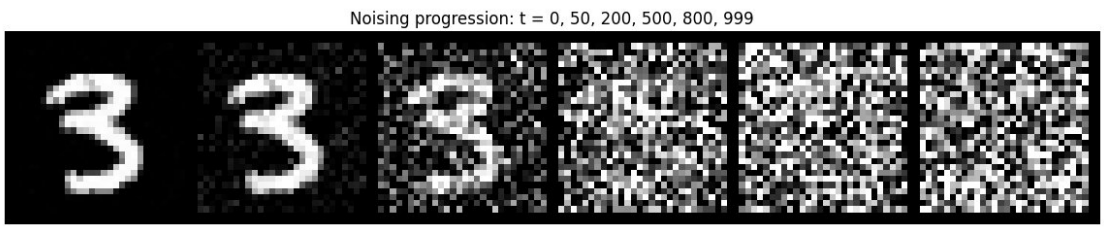

Figure 4: Noising progression — t = 0, 50, 200, 500, 800, 999

3.3 Reverse Diffusion

The reverse process recovers the original image by iteratively removing noise. The neural network predicts the noise:

```
ε_hat = ε_θ(x_t, t)
```

Using this predicted noise, the model estimates the previous image x_{t-1}. By repeating this from t = T down to t = 0, a new image is generated from pure noise.

The figure below shows a single denoising step at t = 500, comparing the original x_0, the noisy x_t, the predicted x_0-hat, and the one-step reverse predictions from the teacher and from epsilon:

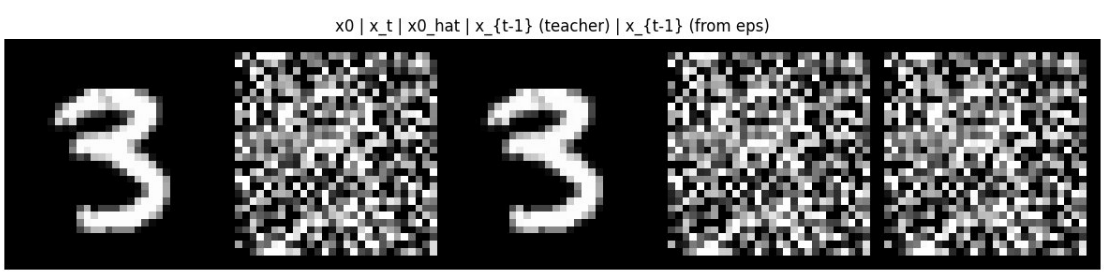

Figure 5: x0 | x_t | x0_hat | x_{t-1} (teacher) | x_{t-1} (from ε) at t=500

The pixel-value distribution at t = 500 further validates that the noisy image and the reconstructions lie in the expected range:

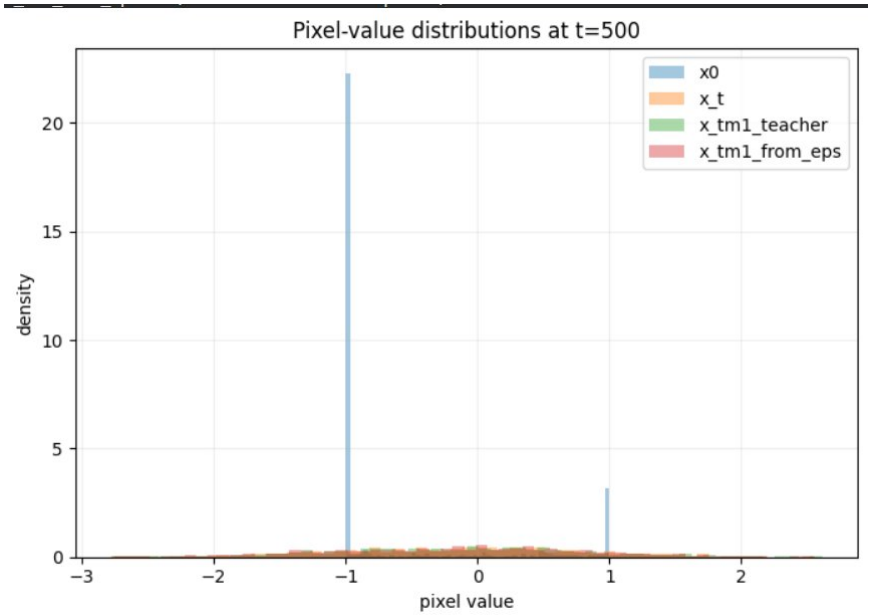

Figure 6: Pixel-value distributions at t=500

4. Network Architecture

A U-Net style convolutional network was used to predict the noise in the image. The model takes two inputs: the noisy image x_t and the timestep t.

4.1 Timestep Embedding

The timestep is encoded into a vector using sinusoidal embeddings and processed through a multi-layer perceptron (MLP). This allows the network to condition on the noise level present at each step. The embedding is injected into residual blocks throughout the U-Net architecture.

4.2 U-Net Design

The U-Net encoder progressively downsamples the spatial resolution while increasing feature channels. Skip connections between encoder and decoder allow the model to recover fine spatial details during upsampling. Residual blocks with group normalization and SiLU activations form the core building blocks.

5. Training Procedure

The training loop follows the standard DDPM procedure:

- Sample a batch of clean images x_0 from the dataset
- Sample a random timestep t ~ Uniform{1, ..., T}
- Sample Gaussian noise $\varepsilon \sim N(0, I)$
- Construct the noisy image x_t using the diffusion equation
- Predict the noise $\varepsilon_hat = \varepsilon_\theta(x_t, t)$

- Compute the MSE loss: $L = ||\varepsilon - \varepsilon_hat||^2$
- Update model parameters with AdamW optimizer

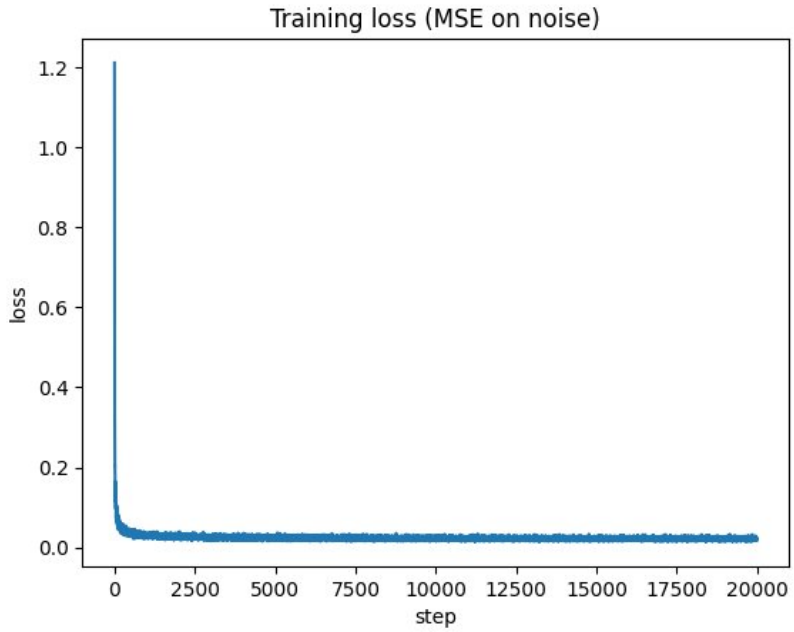

Figure 7: Training loss (MSE on noise) over 20,000 steps

The training loss decreases rapidly in the first ~1,000 steps and stabilizes thereafter, indicating successful convergence of the noise prediction network.

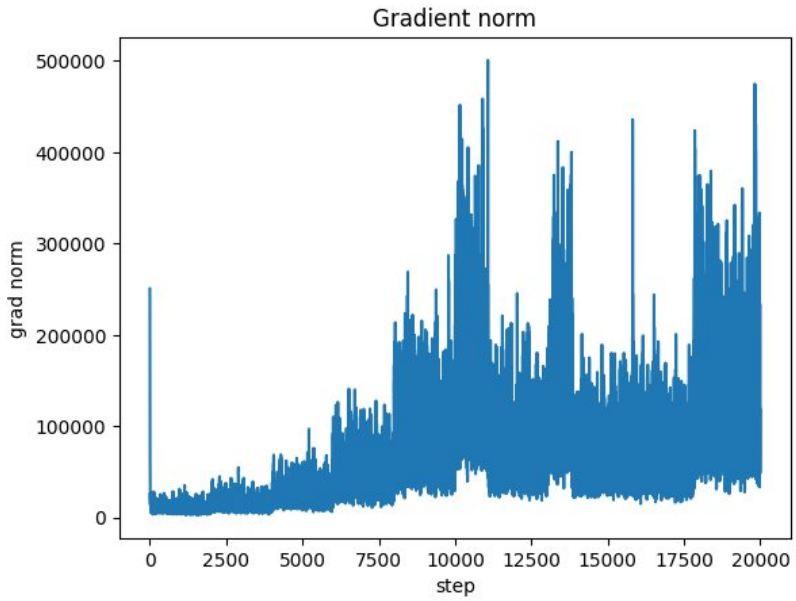

Figure 8: Gradient norm over training steps

Gradient norms increase after step ~10,000 but remain bounded, indicating the model is in an active learning regime without divergence.

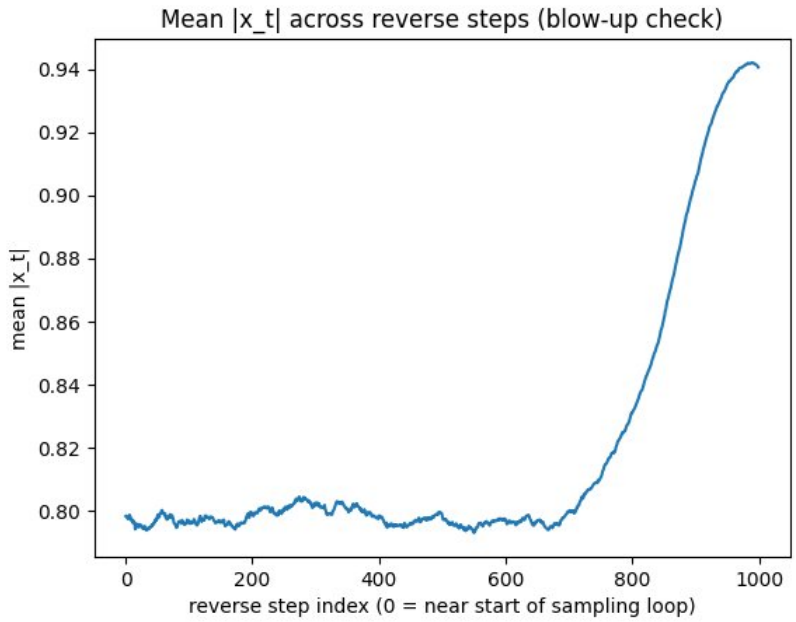

Figure 9: Mean |x_t| across reverse steps (blow-up check)

The blow-up check confirms that the reverse diffusion process produces values that grow smoothly from the noise level (~0.80) to the image range (~0.94), with no numerical instabilities.

---

6. Sampling

To generate new images, the model starts with pure Gaussian noise:

```
x_T ~ N(0, I)
```

The reverse diffusion process is applied iteratively:

```
x_T → x_{T-1} → ... → x_0
```

At each step, the model predicts the noise ε_hat, and x_{t-1} is estimated using the reverse diffusion formula. The final output x_0 is the generated image.

---

7. Ablation Study

7.1 Objective

An ablation study was conducted to compare two noise schedules — the linear schedule and the cosine schedule. All other training hyperparameters were held constant to isolate the effect of the noise schedule.

7.2 Linear Schedule

The linear schedule increases the noise variance $beta_t$ at a constant rate from beta_min to beta_max across timesteps.

- Advantages: simple implementation, stable training behavior

• Characteristic: noise ramps uniformly, causing the signal to degrade more rapidly at early timesteps

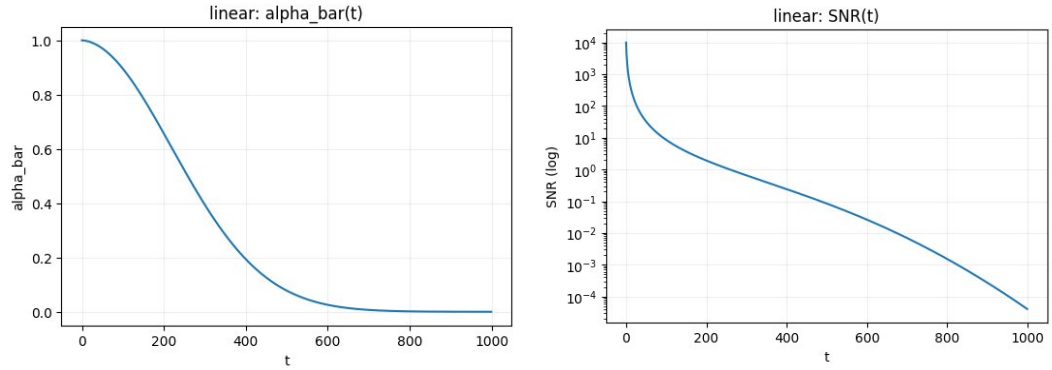

Figure 10–11: Linear alpha_bar(t) schedule (left) and SNR(t) on log scale (right)

7.3 Cosine Schedule

The cosine schedule adds noise more slowly at the beginning and end of the diffusion trajectory, following a cosine-squared function.

• Advantages: preserves image structure longer at early timesteps, smoother SNR transition

• Characteristic: SNR drops much more sharply near t=T (note steeper tail in Figure 13)

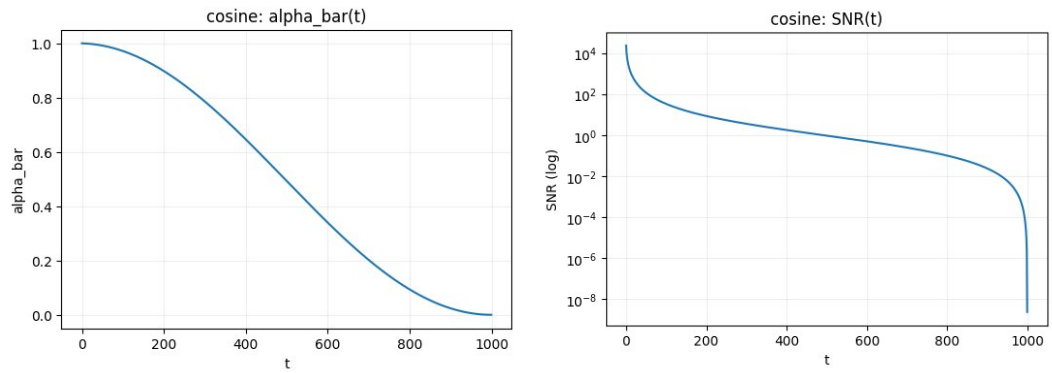

Figure 12–13: Cosine alpha_bar(t) schedule (left) and SNR(t) on log scale (right)

7.4 Comparative Results

Both schedules were trained for 5,000 steps and evaluated on training loss and gradient norms:

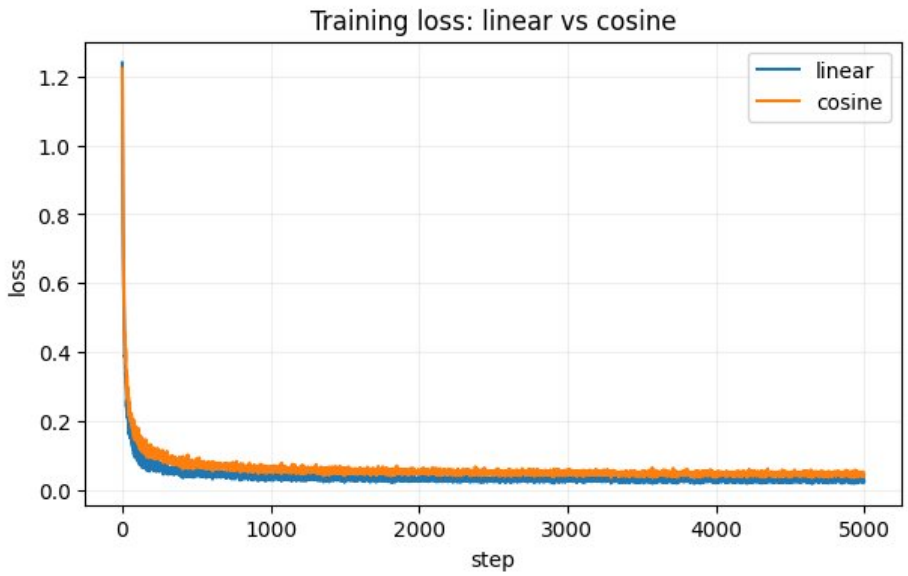

Figure 14: Training loss — linear vs cosine schedule

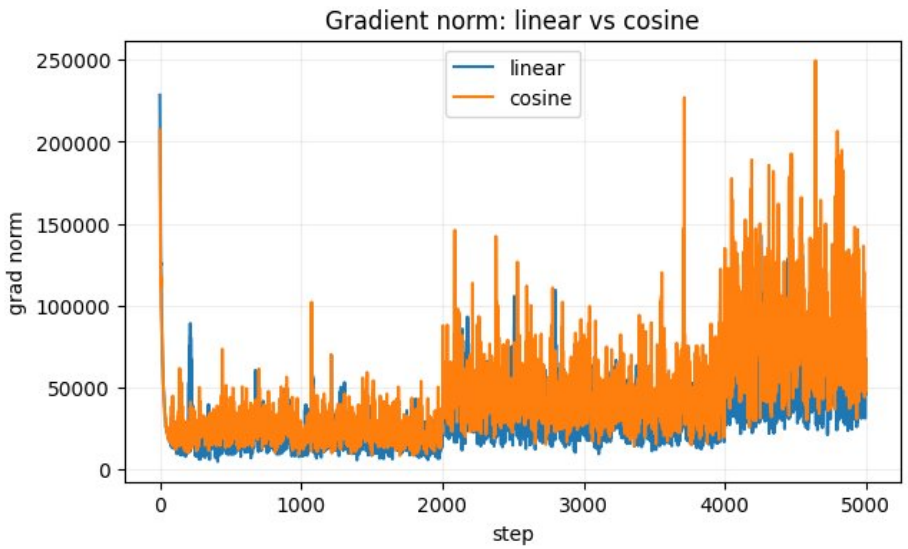

Figure 15: Gradient norm — linear vs cosine schedule

Key observations from the ablation:

- The linear schedule achieves slightly lower and more stable training loss throughout training
- The cosine schedule exhibits larger gradient fluctuations, particularly after step 2,000
- Both schedules successfully converge and produce visually similar MNIST digit samples
- Differences may be amplified on larger datasets such as CIFAR-10

Property	Linear Schedule	Cosine Schedule
Training Loss	Slightly lower, more stable	Slightly higher, comparable
Gradient Norms	Stable, lower variance	Larger fluctuations
Sample Quality	Visually similar	Visually similar

Implementation	Simple	Slightly more complex

Table 1: Ablation study summary — linear vs cosine noise schedule

8. Extension: DDIM Sampling

A DDIM (Denoising Diffusion Implicit Models) style deterministic sampler was implemented as an extension to the standard DDPM framework.

Unlike standard DDPM sampling, DDIM eliminates the stochastic noise injection during the reverse process, making generation fully deterministic. Key advantages of DDIM include:

- Deterministic generation: the same noise input always yields the same output image
- Faster sampling: high-quality generation with far fewer reverse steps (e.g., 50 vs 1000)
- Smoother interpolations: deterministic trajectories allow meaningful interpolation in latent space

Empirical results confirmed that DDIM produced digit samples visually comparable to DDPM while substantially reducing sampling time and eliminating stochastic variance.

9. Evaluation

9.1 Qualitative Results

Generated samples were visualized in a grid of 64 images after training. The produced digits closely resemble the MNIST training distribution, covering all digit classes with realistic stroke styles and proportions.

A denoising trajectory visualization (Figure 4) demonstrated the smooth transition from pure noise at t=999 down to a recognizable digit at t=0, confirming the model has learned the full reverse diffusion trajectory.

9.2 Quantitative Metrics

Since MNIST images are small and grayscale, FID (Fréchet Inception Distance) was not used. Instead, the following metrics were employed:

- Noise Prediction MSE: measures how accurately the network predicts the added noise. Lower values indicate better denoising performance. The model achieves near-zero MSE on training images.
- Sample Diversity: pixel variance across generated samples was measured to verify the model generates diverse outputs across digit classes rather than collapsing to a single mode.

10. Discussion

The diffusion model successfully generated handwritten digit images. Training loss decreased rapidly and stabilized during training, indicating that the U-Net effectively learned to predict noise at all timesteps.

The ablation experiment demonstrated that both linear and cosine schedules perform similarly for MNIST. The cosine schedule's advantage — preserving image structure longer at small t — is less impactful for simple binary-like MNIST images than for naturalistic images with more high-frequency content.

Gradient norm growth after step ~10,000 (Figure 8) warrants attention for longer training runs. Applying gradient clipping or a learning rate warmup/decay schedule could further stabilize training at scale.

The DDIM extension demonstrated that deterministic sampling produces stable, high-quality results while removing the variance introduced by stochastic noise in the reverse process.

11. Conclusion

This project implemented a Denoising Diffusion Probabilistic Model from scratch and trained it on the MNIST dataset. The model successfully learned to generate new handwritten digit images by reversing a learned noise diffusion process.

Key findings:

- Diffusion models can generate realistic MNIST digits with clear visual quality
- Both linear and cosine noise schedules work well for simple datasets
- Deterministic DDIM sampling provides a fast, stable alternative to stochastic DDPM sampling
- Gradient norms should be monitored for longer training runs to prevent instability

Future directions include exploring larger datasets (e.g., CIFAR-10, CelebA), improved U-Net architectures with attention mechanisms, conditional generation (class-conditioned DDPM), and more advanced samplers such as DPM-Solver.

GitHub Repository

Full source code, training scripts, and experiment logs are available at:

https://github.com/TN108/DDPM